
AAD MODULE 3 SELF LEARNING

Greedy vs Dynamic Programming

Sithaara Jubab Roshan
S5AD
67
U2408077

CORE PARADIGMS

Understanding the Approaches

Philosophy & Decision Making

Greedy Paradigm

Immediate Local Choice: Makes the locally optimal choice at each step without considering future consequences.

- Commits irreversibly to decisions once made
- Never backtracks or re-evaluates previous stages
- Extremely fast and minimal memory footprint
- Requires proof that local choices lead to global optimum

Dynamic Programming

Systematic Subproblem Search: Evaluates all possible decision paths by combining solutions to subproblems.

- Considers future consequences before finalizing choices
- Stores intermediate results (Memorization / Tabulation)
- Higher time and memory overhead than Greedy
- Guarantees mathematical global optimality

Key Theoretical Prerequisites



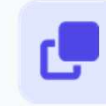
Greedy Choice Property

A globally optimal solution can be arrived at by making locally optimal (greedy) choices. **Required strictly by Greedy algorithms.**



Optimal Substructure

An optimal solution to the problem contains optimal solutions to its subproblems. **Required by both Greedy and Dynamic Programming.**



Overlapping Subproblems

The recursive space contains small subproblems solved repeatedly. Storing solutions avoids recalculation. **Required strictly by DP.**

The Classic Boundary: Knapsack

Greedy Solution

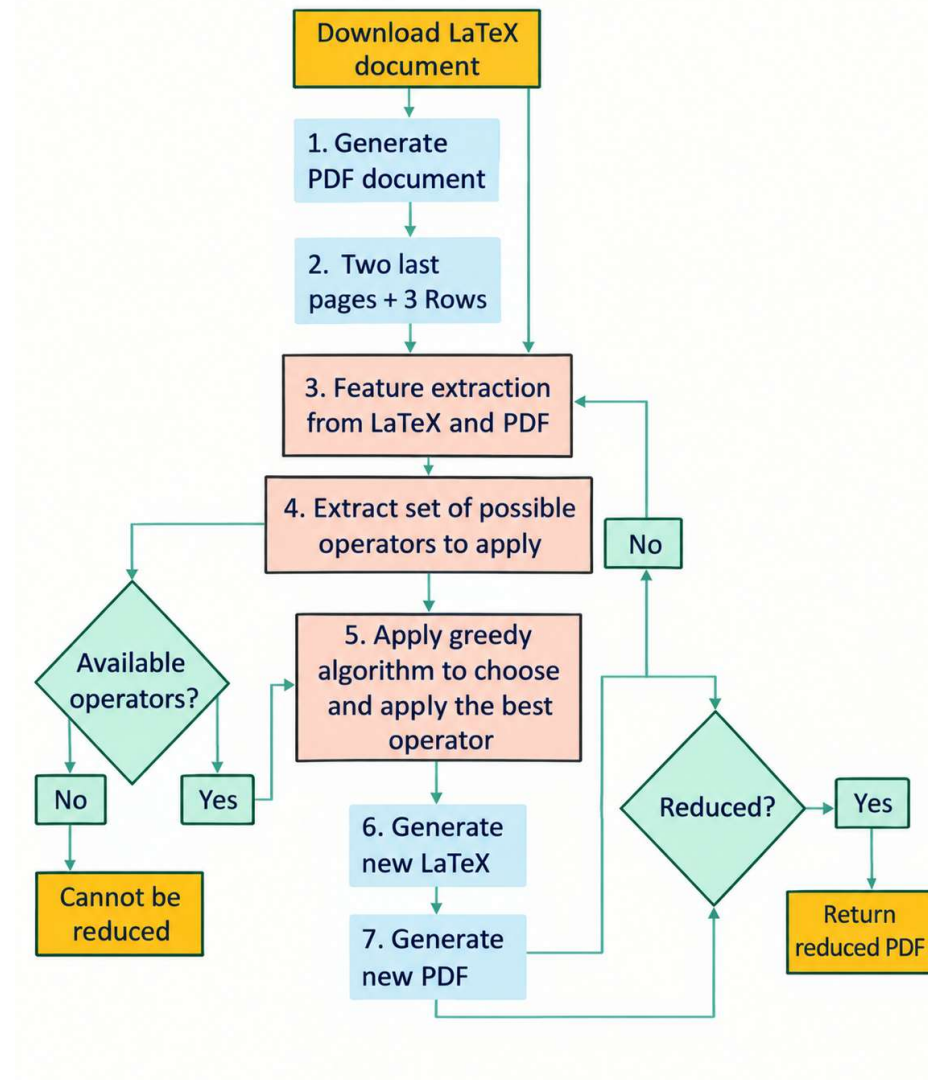
Fractional Knapsack

Items can be divided. We sort by value-to-weight ratio and take as much of the highest-ratio item as possible. The greedy choice property holds perfectly!

DP Solution

0/1 Knapsack

Items are indivisible (take all or nothing). A greedy choice based on value-to-weight ratio fails because taking a high-ratio item may waste precious remaining capacity. DP must evaluate all combinations.



Complexity Trade-offs

$O(N \log N)$

Greedy vs DP $O(N \times W)$

Efficiency vs Precision

Greedy algorithms prioritize execution speed and negligible auxiliary memory space by committing to immediate local decisions without looking back.

Dynamic Programming willingly trades higher computational time and table memory $O(N \cdot \text{State})$ in order to explore state transitions and guarantee global mathematical optimality.

Architectural Comparison

Parameter	Greedy Approach	Dynamic Programming
Decision Strategy	Single locally optimal choice at each step	Evaluates all subproblem choices systematically
Subproblem Overlap	Not required (tree-like decision structure)	Mandatory (overlapping subproblems cached)
Reconsideration	Never backtracks or alters past decisions	Combines past subproblem solutions iteratively
Space Complexity	Minimal: $O(1)$ auxiliary space usually	Moderate to High: $O(N)$ to $O(N^2)$ state tables
Optimality Guarantee	Guaranteed only if Greedy Choice Property holds	Always guaranteed for valid formulations

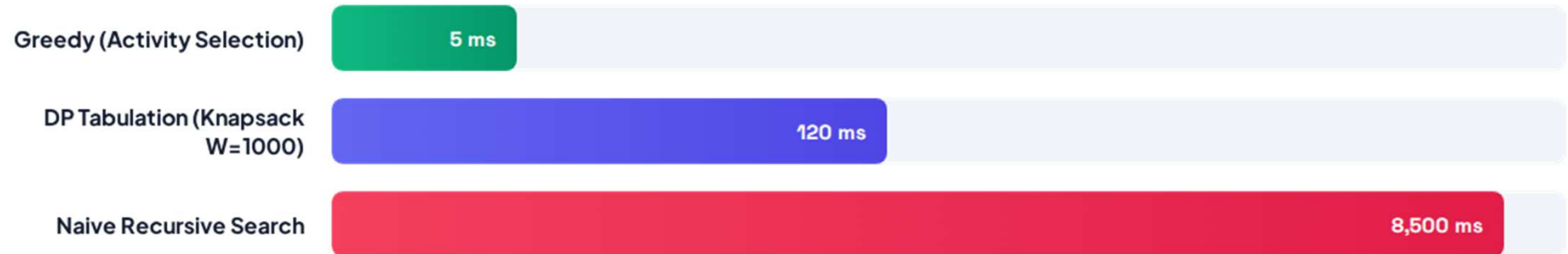
When Greedy Fails: Coin Change

- ✓ **Standard Currency (1, 5, 10, 25):** Greedy works perfectly! Taking the largest coin denomination first yields the minimum total coins.
- ✗ **Custom Coins (1, 3, 4) for Target 6:**
 - Greedy choice: $4 + 1 + 1 = 3$ coins (Suboptimal)
 - DP choice: $3 + 3 = 2$ coins (Globally Optimal)
- 💡 **Key Takeaway:** Finding a single small counterexample instantly disproves a greedy approach, forcing a shift to Dynamic Programming.

5-Step Decision Framework



Execution Time Comparison



Benchmarking Takeaway: Greedy provides sub-millisecond execution when applicable. Dynamic Programming tames exponential recursive explosions into predictable polynomial runtimes.

Conclusion



Greedy is faster when it's correct, and just wrong when it isn't. If you cannot prove local choice optimality, trust Dynamic Programming.

